VERSIONING/DOCUMENTING ASP.NET CORE RESTFUL SERVICES

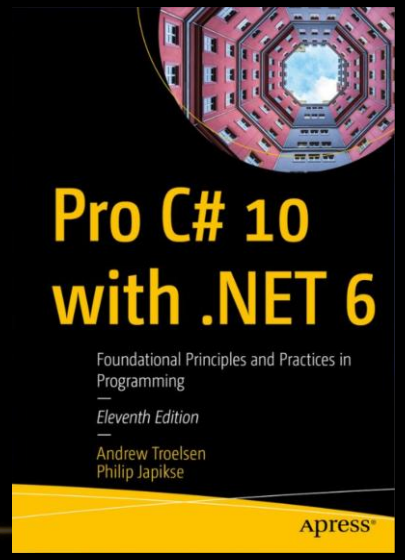
Philip Japikse (@skimedic)
skimedic@outlook.com
CTO, Author, Teacher
Microsoft MVP, ASPInsider, PST, PSM II, PSD

https://github.com/skimedic/presentations

All slides copyright Philip Japikse http://www.skimedic.com

Phil.About()

- CTO/Chief Architect, Pintas & Mullins
- Author: Apress.com (http://bit.ly/apressbooks)
- Professional Scrum Trainer (PSF, PSD)
- Speaker: http://www.skimedic.com/blog/page/Abstracts.aspx
- Microsoft MVP, ASPInsider, PST, PSM II, PSD
- Founder, Agile Conferences, Inc.
  - http://www.cincydeliver.org
- President, Cincinnati .NET User's Group



All slides copyright Philip Japikse http://www.skimedic.com

VERSIONING SERVICES

All slides copyright Philip Japikse http://www.skimedic.com

DO YOU NEED TO VERSION?

- Your API is Public
- Your API needs updating
- Your API has more than one client OR you plan on adding more
- You want to plan for the future (but not gold plate)

- Clients need to count on services being stable over time
- Business needs to add new features and make changes

All slides copyright Philip Japikse http://www.skimedic.com

VERSIONING OPTIONS

All slides copyright Philip Japikse http://www.skimedic.com

VERSION FORMATS

- Services are versioned using the Major.Minor versioning scheme
  - 1.0, 2.0
- Services can opt for only the Major version - the ".0" is implied
  - v1 => v1.0
- Status (RC, Alpha, Beta, etc.) can be specified after the Minor version
  - 1.0-Alpha
- Grouping using YYYY-MM-DD
  - 2018-06-12.1.0-RC

https://github.com/Microsoft/api-guidelines/blob/vNext/Guidelines.md

All slides copyright Philip Japikse http://www.skimedic.com

VERSIONING OPTIONS

- Embed the version after the service root
  - https://www.skimedic.com/api/v1.0/classes
- Use a query string parameter
  - https://www.skimedic.com/api/classes?api-version=1.0
- HTTP Headers (Not compliant with MS REST Guidelines)
  - api-version:2.0
- Use Media Type (Not compliant, but generally accepted)
  - Content-Type: application/json;v=2.0 (or Accept)
    - Content-Type wins if both are present

https://github.com/Microsoft/api-guidelines/blob/vNext/Guidelines.md

All slides copyright Philip Japikse http://www.skimedic.com

GROUP VERSIONING

- Group Versioning is an optional feature
  - Defined using the YYYY-MM-DD format
  - Does not replace the Major.Minor version format
- Allows for logical grouping of API
  - Developers can lookup a single version and use it across related end points
- Can cause confusion due to reusing versions

All slides copyright Philip Japikse http://www.skimedic.com

MICROSOFT REST API GUIDELINES ON VERSIONING

All slides copyright Philip Japikse http://www.skimedic.com

API GUIDANCE

- Present a consistent user experience
- Guarantee stability of the REST APIs
- Do not change names or structures over time

https://github.com/Microsoft/api-guidelines/blob/vNext/Guidelines.md

All slides copyright Philip Japikse http://www.skimedic.com

VERSIONING GUIDANCE

- Be consistent with versioning mechanism (URL v. Query String)
- Indicate deprecated version(s)
- Update versions with breaking changes

- https://github.com/Microsoft/api-guidelines/blob/master/Guidelines.md#12-versioning

All slides copyright Philip Japikse http://www.skimedic.com

ALL REQUIRE UPDATED VERSIONING

- Breaking Changes –
  - Removal or renaming APIs or API parameters
  - Changes in the behavior of an existing API
  - Changes in Error Codes and Fault Contracts

- New Features

- Anything that violates the Principle of Least Astonishment

All slides copyright Philip Japikse http://www.skimedic.com

PRINCIPLE OF LEAST ASTONISHMENT

- If a necessary feature has a high astonishment factor, it may be necessary to redesign the feature - 1984
  - A component of a system should behave in a way that users expect
- For an API, function or method names intuitively match their behavior

https://en.wikipedia.org/wiki/Principle_of_least_astonishment

All slides copyright Philip Japikse http://www.skimedic.com

VERSIONING ASP.NET CORE WEB SERVICES

All slides copyright Philip Japikse http://www.skimedic.com

OVERVIEW

- Add Package Asp.Versioning.Mvc
- In Program.cs
  - Add call to builder.Services.AddApiVersioning().AddMvc()
  - Add call to builder.Services.AddProblemDetails
- Use Attribute Routing for controllers
  - If using URL versioning, update the route
- Add version attributes
- Getting the version requested:
  - HttpContext.GetRequestedApiVersion || Model Binding

All slides copyright Philip Japikse http://www.skimedic.com

ADDING VERSIONING TO CONTROLLERS

- Use the ApiVersion attribute to add versioning

```
//Query String and Media Type
[ApiVersion("2.0")]
[Route("api/helloworld")]
public class HelloWorld2Controller : Controller
{
    …
}
//URL Versioning
[ApiVersion(1.0)]
[Route("api/v{version:apiVersion}/[controller]")]
public class HelloWorldController : Controller
{
    …
}
```

All slides copyright Philip Japikse http://www.skimedic.com

VERSION INTERLEAVING

- Use the ApiVersion/MapToApiVersion attributes to add versioning

```
[ApiVersion("2.0")]
[ApiVersion("3.0")]
[Route("api/v{version:apiVersion}/helloworld")]
public class HelloWorld2Controller : Controller
{
    [HttpGet]
    public IActionResult Get() => "Hello world v2!";

    [HttpGet, MapToApiVersion("3.0")]
    public IActionResult GetV3() => "Hello world v3!";
}
```

All slides copyright Philip Japikse http://www.skimedic.com

DEPRECATING VERSIONS

- Add Deprecated to the ApiVersion attribute

```
[ApiVersion("1.0", Deprecated = true)]
[Route( "api/[controller]" )]
public class HelloWorldController : Controller
{
   //omitted
}
```

All slides copyright Philip Japikse http://www.skimedic.com

ADVERTISE VERSIONS

- Advertise versions outside of the controller heirarchy

```
[AdvertiseApiVersions("2.0","3.0")]
[AdvertiseApiVersions("1.5",Deprecated=true)]
[ApiVersion(1.0)]
[Route( "api/[controller]" )]
public class HelloWorldController : Controller
{
   //omitted
}
```

All slides copyright Philip Japikse http://www.skimedic.com

VERSION NEUTRALITY

- Use the ApiVersionNeutral attribute to expose an endpoint to all versions

```
[ApiVersionNeutral]
[Route("api/v{version:apiVersion}/[controller]/[action]")]
public class HealthController : Controller
{
    [HttpGet]
    public IActionResult Ping(ApiVersion versionFromDI)
    {
       ApiVersion version = HttpContext.GetRequestedApiVersion();
    }
}
```

All slides copyright Philip Japikse http://www.skimedic.com

REQUESTS AND VERSION INFORMATION

- Request Formats (querystring, header, URL):
  - QueryString (?api-version=1.0)
  - URL (api/v1.0/[Controller])
  - Media Type (Content-Type: application/json;v=2.0) || Accept
  - Header – must configure manually with ApiVersionReader
  - Note: All are customizable

- Getting Version requested:
  - HttpContext.GetRequestedApiVersion
  - Model Binding supported in 3.0+

All slides copyright Philip Japikse http://www.skimedic.com

GETTING THE REQUESTED VERSION INFORMATION

- Use the GetRequestedApiVersion or ModelBinding to return the requested version information

```
[ApiVersion("1.0")]
[ApiVersion("2.0")]
[Route("api/v{version:apiVersion}/[controller]/[action]")]
public class DifferentVersionsController : Controller
{
   [HttpGet]
   public string RequestedApiVersion() =>
      JsonConvert.SerializeObject(HttpContext.GetRequestedApiVersion());

   [HttpGet] //3.0+
   public string Get(ApiVersion apiVersion)
      => $"Controller = {GetType().Name}\nVersion = {apiVersion}";
}
```

All slides copyright Philip Japikse http://www.skimedic.com

API VERSIONING OPTIONS

All slides copyright Philip Japikse http://www.skimedic.com

SETTING DEFAULT VERSION FOR REQUESTS

- AssumeDefaultVersionWhenUnspecified
  - Used when adding versioning to an existing API
  - Returned version is configured with ApiVersionSelector

- ApiVersionSelector defines the behavior for unspecified requests.
  - Default – the configured default
  - Constant – always selects the specified version
  - Current Implementation – greatest version number (w/o status)
  - Lowest Implementation – lowest version number (w/o status)

All slides copyright Philip Japikse http://www.skimedic.com

DEFAULT VERSIONS FOR APICONTROLLERS

- DefaultApiVersion – Versions Controllers w/o ApiVersion attribute
  - Configured default value is 1.0
  - Can be set to another value

- Also used with AssumeDefaultVersionWhenUnspecified when Default is the ApiVersionSelector

All slides copyright Philip Japikse http://www.skimedic.com

APIVERSIONREADER

- The **IApiVersionReader** interface defines the behavior of how an API version is read in its raw, unparsed form from the current HTTP request.
- Default version reader handles query string and URL segments
- Can add/update defaults for:
  - QueryStringApiVersionReader
  - HeaderApiVersionReader
  - MediaTypeApiVersionReader
- Allows composition with Combine(versionreaders)

All slides copyright Philip Japikse http://www.skimedic.com

ADDITIONAL SETTINGS

- ReportApiVersion
  - Reports available versions within a controller hierarchy
- Conventions
  - Allows for the construction of API version conventions
- Route Constraint Name
  - Allows for changing the name of the route constraint name from "apiVersion"
- Policies
  - Allows for defining API versioning policies, such as setting sunset dates

All slides copyright Philip Japikse http://www.skimedic.com

ERROR RESPONSES

- Responds with ProblemDetails
  - Requires services.AddProblemDetails()
- Standard errors:
  - 400 – Unspecified version (required, but not specified)
  - 400 – Unsupported version
  - 400 – Invalid version
  - 400 – Ambiguous version
- Can be customized

All slides copyright Philip Japikse http://www.skimedic.com

LIMITING VERSIONING TO API CONTROLLERS

- In projects with mixed API and UI controllers, specify which controllers are versioned:

```
builder.Services.TryAddEnumerable(
  ServiceDescriptor.Transient<IApiControllerSpecification,
  ApiBehaviorSpecification>());
```

- Can also build a custom filter

All slides copyright Philip Japikse http://www.skimedic.com

OPENAPI

All slides copyright Philip Japikse http://www.skimedic.com

SWAGGER/OPENAPI

- Swagger is a language-agnostic spec for describing REST APIs
  - When donated to the OpenAPI Initiative was renamed to OpenAPI
- Provides interactive documentation, client SDK Generation, and API discoverability
- Options for generation of documents in ASP.NET Core
  - Microsoft.AspNetCore.OpenApi
  - Swashbuckle.AspNetCore generates docs
  - Nswag generates docs + client code

All slides copyright Philip Japikse http://www.skimedic.com

API DOCUMENTATION WITH VERSIONING

All slides copyright Philip Japikse http://www.skimedic.com

API DOCUMENTATION

- Microsoft.AspNetCore.OpenApi has replaced Swashbuckle SwaggerGen.
- Add package Asp.Versioning.Mvc.ApiExplorer
- Add .AddApiExplorer() call to AddApiVersioning().AddMvc()
- Minimal configuration:
  - GroupNameFormat – usually "v'VVV" (eg v1.5-Beta)
  - SubstituteApiVersionInUrl = true
  - AddApiVersionParametersWhenVersionNeutral = true

All slides copyright Philip Japikse http://www.skimedic.com

ADDITIONAL CONFIGURATION

- FormatGroupName – provides a callback mechanism to change group name
- SubstitutionFormat – controls how versions are added into URLs
- DefaultApiVersion, AssumeDefaultVersionWhenUnspecified, RouteConstraintName, ApiParameterSource
  - Derive from versioning options and should not be used
- DefaultApiVersionParameterDescription
  - Defaults to “The requested API version”

All slides copyright Philip Japikse http://www.skimedic.com

DOCUMENTING ACTION METHODS

- Produces – content type(s)
- ProducesResponseType – status codes and content type
- EndPointSummary – title/summary for endpoint
- EndPointDescription – details for the endpoint
- ApiExplorerSettings(IgnoreApi=true) – exclude from documentation
- Description – description for parameters

All slides copyright Philip Japikse http://www.skimedic.com

XML COMMENTS

- XML Comments are supported
- Update csproj

```
<GenerateDocumentationFile>true</GenerateDocumentationFile>
<NoWarn>1591</NoWarn> //disable warnings for uncommented methods
```

- Also uses referenced projects (if XML comments are generated)
- NOTE: OpenAPI attributes are better for controllers, XML comments for classes (like entities)

All slides copyright Philip Japikse http://www.skimedic.com

GENERATE THE DOCUMENTS

- Set the version in csproj file

```
<OpenApiGenerateDocumentsOptions>--openapi-version OpenApi3_1
</OpenApiGenerateDocumentsOptions>
```

- Call AddOpenApi()
  - Can also set the version in the options
- For multiple documents, call AddOpenApi("documentName",options)

All slides copyright Philip Japikse http://www.skimedic.com

PRESENT THE DOCUMENTS

- Using Scalar
  - Add package Scalar.AspNetCore
  - For each document, call app.MapScalarApiReference()
    - Add in document name and title
- Using Swashbuckle SwaggerUI
  - Add package Swashbuckle.AspNetCore.SwaggerUI
  - Call app.UseSwaggerUI
  - For each document, add SwaggerEndpoint

All slides copyright Philip Japikse http://www.skimedic.com

Contact Me

skimedic@outlook.com
www.skimedic.com/blog
www.twitter.com/skimedic

http://bit.ly/apressbooks

https://github.com/skimedic/presentations

Questions?

Thank You!

https://github.com/skimedic/presentations

All slides copyright Philip Japikse http://www.skimedic.com